

ICD-10-CM Data & RAG Pipeline: A Deep Dive

Table of Contents

- 1. What Are ICD-10 Codes?
- 2. Reading an ICD-10 Code (Anatomy of a Code)
- 3. Our Source Files
- 4. Tabular File Structure
- 5. Index File Structure
- 6. The RAG Pipeline: From XML to Doctor's Screen
 - ? Step 1: ETL ? Parse & Enrich
 - ? Step 2: Embed & Index
 - ? Step 3: Runtime Retrieval
 - ? Step 4: Re-Ranking
 - ? Step 5: Specificity Check
- 7. How It All Connects

What Are ICD-10 Codes?

ICD-10-CM stands for International Classification of Diseases, 10th Revision, Clinical Modification. It's the standardized system used in the United States (and many countries) to describe every medical diagnosis a patient can have. Think of it as a universal language for diseases ? instead of one doctor writing "heart attack" and another writing "coronary event," they both write I21.9 and the entire healthcare system knows exactly what happened.

Why ICD-10 Codes Exist

Purpose	How ICD-10 helps
Billing	Insurance companies pay based on codes, not fre...
Statistics	Governments track disease prevalence (e.g., COV...
Communication	A doctor in Tokyo and one in New York share the...
Legal records	Court cases, disability claims, and death certi...
Research	Clinical trials filter patients by diagnosis codes

Scale

- ? ~72,000 codes in ICD-10-CM 2026
- ? Organized into 21 chapters (body systems + special categories)
- ? Updated annually every October by CMS (Centers for Medicare & Medicaid Services)

Real-World Examples

Code	What it means	When a doctor uses it
J06.9	Acute upper respiratory infection, unspecified	"Common cold"
I21.9	Acute myocardial infarction, unspecified	"Heart attack"
E11.9	Type 2 diabetes mellitus without complications	"Sugar is high"
G44.2	Tension-type headache	"Stress headache"

M54.5	Low back pain	"My back hurts"
H66.93	Otitis media, unspecified, bilateral	"Ear infection in both ears"

Notice how casual language maps to precise codes. This mapping is the core challenge our RAG system solves.

Reading an ICD-10 Code

Every code has a structure that encodes meaning:

```
I 21 . 0 1
? ? ? ? ?
? ? ? ? ? 5th character: Further specificity
? ? ? ? ? 4th character: Subcategory (type/site)
? ? ? ? ? 2-3rd characters: Category (disease)
? ? ? ? ? 1st character: Chapter (body system)
```

The First Character (Letter) = Chapter/Body System

Letter(s)	Chapter	Body System
A?B	1	Infectious diseases
C?D	2?3	Neoplasms (cancer) / Blood diseases
E	4	Endocrine (diabetes, thyroid)
F	5	Mental/behavioral (depression, anxiety)
G	6	Nervous system (headache, epilepsy)
H	7?8	Eye and Ear
I	9	Circulatory (heart, blood vessels)
J	10	Respiratory (lungs, throat)
K	11	Digestive (stomach, liver)
L	12	Skin
M	13	Musculoskeletal (bones, joints, back)
N	14	Genitourinary (kidneys, reproductive)
O	15	Pregnancy/childbirth
P	16	Perinatal (newborn conditions)
Q	17	Congenital (birth defects)
R	18	Symptoms not elsewhere classified
S?T	19	Injuries/poisoning
V?Y	20	External causes
Z	21	Health status/contact with services

Example Breakdown: I21.01

I = Chapter 9: Diseases of the circulatory system
21 = Category: Acute myocardial infarction (heart attack)
.0 = Subcategory: ST elevation (STEMI) of anterior wall
1 = Specificity: involving left main coronary artery

Example Breakdown: H66.93

H = Chapter 7-8: Diseases of the ear
66 = Category: Suppurative and unspecified otitis media (ear infection)

.9 = Subcategory: Otitis media, unspecified type
3 = Specificity: bilateral (both ears)

The Hierarchy Matters

Codes are hierarchical ? like a tree:

```
I21      Acute myocardial infarction (the "parent")
??? I21.0  STEMI of anterior wall
?   ??? I21.01 involving left main coronary artery
?   ??? I21.02 involving left anterior descending coronary artery
?   ??? I21.09 involving other coronary artery of anterior wall
??? I21.1  STEMI of inferior wall
??? I21.2  STEMI of other sites
??? I21.3  STEMI of unspecified site
??? I21.4  Non-ST elevation (NSTEMI) myocardial infarction
??? I21.9  Acute myocardial infarction, unspecified ? the "lazy" code
??? I21.A  Other type of myocardial infarction
```

Why this matters for billing: Insurance companies want the most specific code. Using I21.9 ("unspecified") when you know it's I21.01 (anterior wall, left main coronary artery) can lead to claim denials or audit flags. Our system's Specificity Check (Step 5) catches this.

Our Source Files

We use two official XML files from CMS.gov (Centers for Medicare & Medicaid Services):

File	Size	Lines	Purpose
icd10cm_tabular_2026.xml	9.3 MB	243,834	The authority ? defines what each code means
icd10cm_index_2026.xml	9.2 MB	322,888	The lookup guide ? maps everyday terms to codes

Think of it like a dictionary:

? Tabular = looking up a word by its spelling ? get the definition

? Index = looking up "that thing where your chest hurts" ? get pointed to the right word

Both are needed because doctors say "ear infection" (Index language), but we need to retrieve "H66.93 - Otitis media, unspecified, bilateral" (Tabular language).

The Tabular File (icd10cm_tabular_2026.xml)

What It Is

The Tabular file is the official code classification. It contains every ICD-10-CM code organized by chapter ? section ? code, with definitions, synonyms, exclusions, and usage rules.

Overall Structure

```
<?xml version="1.0" encoding="utf-8"?>
<ICD10CM.tabular>
  <version>2026</version>
```

```

<introduction>...</introduction>      <!-- Instructions on how to use the file -->
<chapter>                               <!-- Repeats 21 times (one per chapter) -->
  <name>1</name>
  <desc>Certain infectious and parasitic diseases (A00-B99)</desc>
  <includes>...</includes>
  <excludes1>...</excludes1>
  <excludes2>...</excludes2>
  <sectionIndex>...</sectionIndex>    <!-- Table of contents for this chapter -->
  <section id="A00-A09">               <!-- Repeats for each code range -->
    <desc>Intestinal infectious diseases (A00-A09)</desc>
    <diag>                             <!-- The actual codes ? nested recursively -->
      <name>A00</name>
      <desc>Cholera</desc>
      <diag>                             <!-- Child code -->
        <name>A00.0</name>
        <desc>Cholera due to Vibrio cholerae 01, biovar cholerae</desc>
      </diag>
    </diag>
  </section>
</chapter>
</ICD10CM.tabular>

```

The Element ? The Heart of the File

Every ICD-10 code is a element. They nest inside each other to form the hierarchy:

```

<diag>                                   <!-- Level 1: Category (3-char) -->
  <name>I21</name>
  <desc>Acute myocardial infarction</desc>
  <includes>
    <note>cardiac infarction</note>
    <note>coronary (artery) embolism</note>
    <note>coronary (artery) occlusion</note>
    <note>coronary (artery) rupture</note>
    <note>coronary (artery) thrombosis</note>
    <note>infarction of heart, myocardium, or ventricle</note>
    <note>myocardial infarction specified as acute or with a stated
      duration of 4 weeks (28 days) or less from onset</note>
  </includes>
  <useAdditionalCode>
    <note>code, if applicable, to identify:</note>
    <note>exposure to environmental tobacco smoke (Z77.22)</note>
    <note>history of tobacco dependence (Z87.891)</note>
    <note>tobacco dependence (F17.-)</note>
    <note>tobacco use (Z72.0)</note>
  </useAdditionalCode>
  <excludes2>
    <note>old myocardial infarction (I25.2)</note>
    <note>postmyocardial infarction syndrome (I24.1)</note>
    <note>subsequent type 1 myocardial infarction (I22.-)</note>
  </excludes2>
  <diag>                                   <!-- Level 2: Subcategory (4-char) -->
    <name>I21.0</name>
    <desc>ST elevation (STEMI) myocardial infarction of anterior wall</desc>
    <inclusionTerm>
      <note>Type 1 ST elevation myocardial infarction of anterior wall</note>
    </inclusionTerm>
    <diag>                                   <!-- Level 3: Specific (5+ char) -->
      <name>I21.01</name>
      <desc>...involving left main coronary artery</desc>
    </diag>
    <diag>
      <name>I21.02</name>
      <desc>...involving left anterior descending coronary artery</desc>
    </diag>
  </diag>

```

```

    </diag>
  </diag>
</diag>

```

Every Element Explained

XML Element	What It Means	Example	Why We Care
	The ICD-10 code itself	I21.01	This is what gets billed and stored in the EHR
	Official description of the code	"Acute myocardial infarction"	Used for display to the doctor
	Other conditions that fall under this code	"cardiac infarction", "coronary thrombosis"	Critical for embeddings ? these are official synonyms
	"Also known as" ? alternative clinical descriptions	"Type 1 ST elevation myocardial infarction"	Critical for embeddings ? more synonyms the doctor might use
	"NOT coded here!" ? conditions that are NOT included	"type 1 diabetes mellitus (E10.-)" and "old myocardial infarction (I25.2)"	Used for validation ? if both appear, something is wrong
	"Not included here" ? conditions that are related but not coded	"old myocardial infarction (I25.2)" and "tobacco use (Z72.0)"	Informational ? tells the doctor about related conditions
	"You should ALSO code..." ? conditions that should be coded	"tobacco use (Z72.0)" with heart attack	Passed to note generation so the LLM can suggest coding
	"Code THIS first, then this one" ? sequencing	"HIV disease (B20)" must come before "AIDS (Z86.0)"	Passed to note generation for proper sequencing

Real Example: Type 2 Diabetes (E11)

```

<diag>
  <name>E11</name>
  <desc>Type 2 diabetes mellitus</desc>
  <includes>
    <note>diabetes (mellitus) due to insulin secretory defect</note>
    <note>diabetes NOS</note>
    <note>insulin resistant diabetes (mellitus)</note>
  </includes>
  <useAdditionalCode>
    <note>code to identify control using:</note>
    <note>injectable non-insulin antidiabetic drugs (Z79.85)</note>
    <note>insulin (Z79.4)</note>
    <note>oral antidiabetic drugs (Z79.84)</note>
  </useAdditionalCode>
  <excludes1>
    <note>diabetes mellitus due to underlying condition (E08.-)</note>
    <note>drug or chemical induced diabetes mellitus (E09.-)</note>
    <note>gestational diabetes (O24.4-)</note>
    <note>neonatal diabetes mellitus (P70.2)</note>
    <note>type 1 diabetes mellitus (E10.-)</note>
  </excludes1>
  <diag>
    <name>E11.0</name>
    <desc>Type 2 diabetes mellitus with hyperosmolarity</desc>
    <diag>
      <name>E11.00</name>
      <desc>Type 2 diabetes mellitus with hyperosmolarity without
        nonketotic hyperglycemic-hyperosmolar coma (NKHHC)</desc>
    </diag>
    <diag>
      <name>E11.01</name>
      <desc>Type 2 diabetes mellitus with hyperosmolarity with coma</desc>
    </diag>
  </diag>
</diag>

```

What this tells us:

- ? "diabetes NOS" (Not Otherwise Specified) maps to E11 ? so when a doctor says "the patient is diabetic" without specifying type, it's E11
- ? If a patient is on insulin for Type 2, you ALSO code Z79.4

- ? Type 1 diabetes (E10) can NEVER be coded at the same time (Excludes1)
- ? Gestational diabetes is completely separate (O24.4-)

Real Example: Laterality ? Otitis Media (H66.9)

```
<diag>
  <name>H66.9</name>
  <desc>Otitis media, unspecified</desc>
  <inclusionTerm>
    <note>Otitis media NOS</note>
    <note>Acute otitis media NOS</note>
    <note>Chronic otitis media NOS</note>
  </inclusionTerm>
  <useAdditionalCode>
    <note>code for any associated perforated tympanic membrane (H72.-)</note>
  </useAdditionalCode>
</diag>
<diag>
  <name>H66.90</name>
  <desc>Otitis media, unspecified, unspecified ear</desc>
</diag>
<diag>
  <name>H66.91</name>
  <desc>Otitis media, unspecified, right ear</desc>
</diag>
<diag>
  <name>H66.92</name>
  <desc>Otitis media, unspecified, left ear</desc>
</diag>
<diag>
  <name>H66.93</name>
  <desc>Otitis media, unspecified, bilateral</desc>
</diag>
</diag>
```

What this tells us:

- ? If a doctor says "ear infection in both ears," the most specific code is H66.93 (bilateral)
- ? Using H66.9 ("unspecified") when you know it's bilateral is less specific than it should be
- ? Our Specificity Check step will catch H66.9 and prompt: "Did you mean bilateral (H66.93)?"

The Index File (icd10cm_index_2026.xml)

What It Is

The Index file is the lookup guide ? organized alphabetically by disease term, not by code. When a doctor says "diabetes with kidney problems," you look it up in the Index and it points you to the exact code (E11.22).

This is the same file that human medical coders use when they can't remember a code ? they look up the condition by name.

Overall Structure

```
<?xml version="1.0" encoding="utf-8"?>
<ICD10CM.index>
  <version>2026</version>
  <title>ICD-10-CM INDEX TO DISEASES and INJURIES</title>
  <letter>                                <!-- Repeats A-Z -->
```

```

<title>A</title>
<mainTerm>                                <!-- One entry per disease/condition -->
  <title>Aarskog's syndrome</title>
  <code>Q87.19</code>
</mainTerm>
<mainTerm>
  <title>Abandonment</title>
  <see>Maltreatment</see>                <!-- Cross-reference -->
</mainTerm>
...
</letter>
</ICD10CM.index>

```

Elements Explained

XML Element	What It Means	Example
	A top-level disease/condition entry	"Diabetes, diabetic"
	The term itself (what you look up)	"Ear infection"
	The ICD-10 code this term maps to	E11.9
	A sub-modifier (narrows the condition)	"with cataract" ? E11.36
	Further narrowing	"coma due to hypoglycemia" ? E11.641
	Even more specific	Rare; deepest nesting
	Non-essential modifier ? parenthetical text tha	"(mellitus) (sugar)" after "Diabetes"
	"Don't look here ? go look at THIS term instead	"Abandonment" ? see "Maltreatment"
	"Also check this related term"	"Abdomen" ? see also "condition"

Real Example: Diabetes in the Index

This is how a medical coder would find "diabetes with kidney problems":

```

<mainTerm>
  <title>Diabetes, diabetic<nemod>(mellitus) (sugar)</nemod></title>
  <code>E11.9</code>                        <!-- Default: Type 2, no complications -->
  <term level="1">
    <title>with</title>
    <term level="2">
      <title>amyotrophy</title>
      <code>E11.44</code>
    </term>
    <term level="2">
      <title>arthropathy NEC</title>
      <code>E11.618</code>
    </term>
    <term level="2">
      <title>autonomic<nemod>(poly)</nemod>neuropathy</title>
      <code>E11.43</code>
    </term>
    <term level="2">
      <title>cataract</title>
      <code>E11.36</code>
    </term>
    <term level="2">
      <title>chronic kidney disease</title>
      <code>E11.22</code>                    <!-- ? Found it! -->
    </term>
    <term level="2">
      <title>coma due to</title>
      <term level="3">
        <title>hyperosmolarity</title>
        <code>E11.01</code>
      </term>
    </term>
  </term>

```

```

</term>
<term level="3">
  <title>hypoglycemia</title>
  <code>E11.641</code>
</term>
<term level="3">
  <title>ketoacidosis</title>
  <code>E11.11</code>
</term>
</term>
</term>
</mainTerm>

```

Reading this like a lookup table:

Look up...	Follow path...	Get code
"Diabetes" (plain)	mainTerm ? code	E11.9
"Diabetes with cataract"	mainTerm ? "with" ? "cataract"	E11.36
"Diabetes with kidney disease"	mainTerm ? "with" ? "chronic kidney disease"	E11.22
"Diabetic coma from low sugar"	mainTerm ? "with" ? "coma due to" ? "hypoglyc	E11.641

The Element ? Non-Essential Modifiers

```

<title>Diabetes, diabetic<nemod>(mellitus) (sugar)</nemod></title>

```

The text inside is parenthetical ? it's there for clarity but doesn't change the code. "Diabetes," "Diabetes mellitus," and "Diabetes sugar" all point to the same code. This tells our parser that "(mellitus)" and "(sugar)" are synonym fragments we should include in the embedding text.

The Element ? Cross-References

```

<mainTerm>
  <title>Abandonment</title>
  <see>Maltreatment</see>
</mainTerm>

```

This means: "If you're looking for 'Abandonment,' the codes are under 'Maltreatment' instead." There is no code directly here ? you must follow the reference. Our parser notes these for completeness but they don't produce embeddings directly.

The Element

```

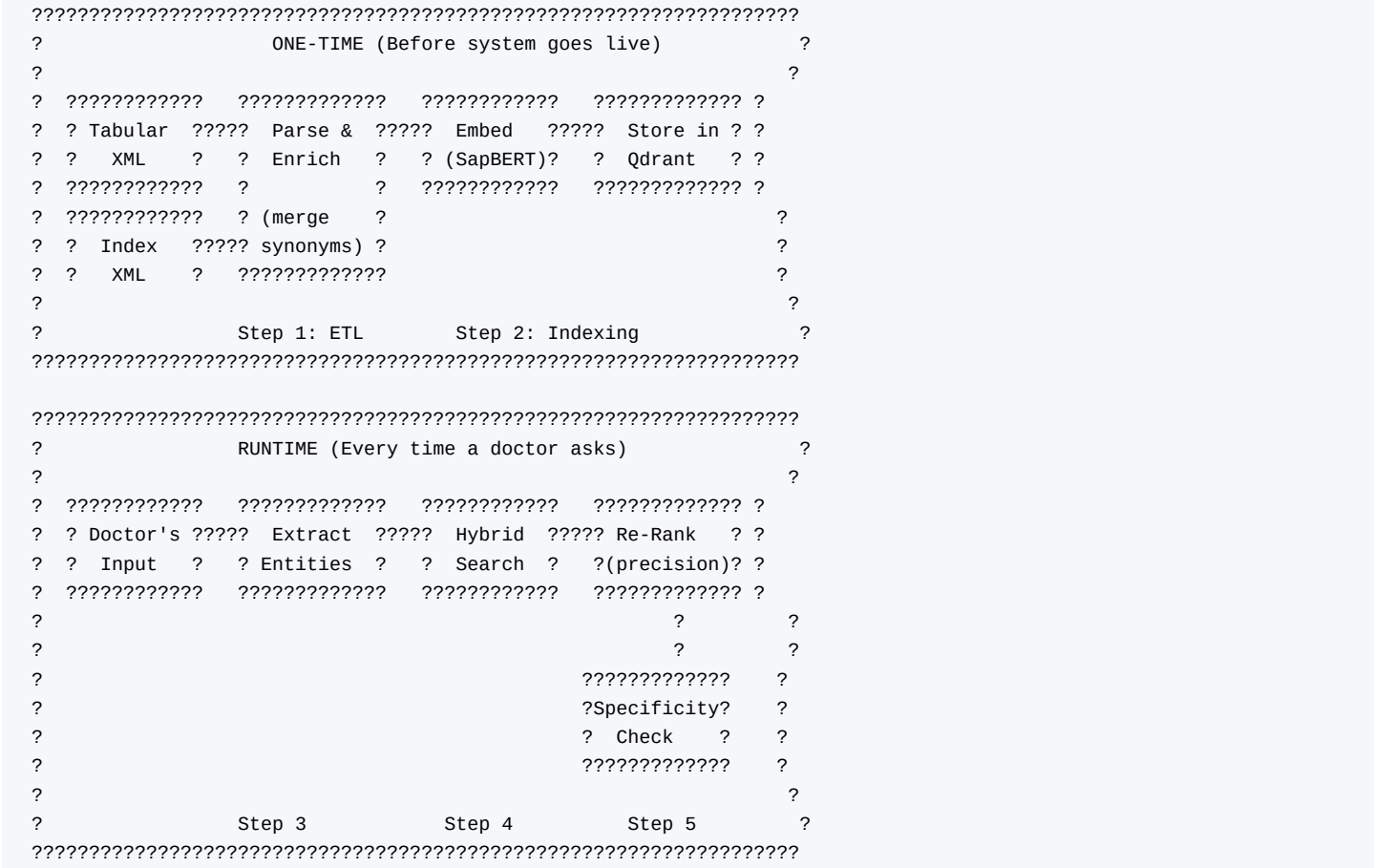
<mainTerm>
  <title>Abdomen, abdominal</title>
  <seeAlso>condition</seeAlso>
  <term level="1">
    <title>acute</title>
    <code>R10.0</code>
  </term>
</mainTerm>

```

This means: "There are some codes here, BUT also check under 'condition' for more." Unlike , there ARE codes at this entry ? is supplementary.

The RAG Pipeline: From XML to Doctor's Screen

Now that you understand the data, here's how we turn 18.5 MB of XML into instant, accurate ICD-10 suggestions.



Step 1: ETL ? Parse & Enrich

Goal: Turn 18.5 MB of nested XML into ~72,000 flat, self-contained documents ready for embedding.

Files involved:

- ? src/mednote/rag/etl/parser.py ? parses the Tabular XML
- ? src/mednote/rag/etl/index_parser.py ? parses the Index XML
- ? src/mednote/rag/etl/metadata.py ? adds demographic filters
- ? src/mednote/rag/etl/export.py ? writes JSONL output
- ? scripts/run_etl.py ? orchestrates the full pipeline

Sub-Step 1a: Parse the Tabular XML

We recursively walk the tree, building the hierarchy path as we go:

```
def parse_icd10_tabular(xml_path: str) -> list[ICD10Code]:
    """Walk: ICD10CM.tabular ? chapter ? section ? diag (recursive)"""
```

Input: A nested element like:

```
<name>H66.9</name>
<desc>Otitis media, unspecified</desc>
<inclusionTerm>
    <note>Otitis media NOS</note>
```

```

    <note>Acute otitis media NOS</note>
</inclusionTerm>
<diag><name>H66.90</name><desc>...unspecified ear</desc></diag>
<diag><name>H66.91</name><desc>...right ear</desc></diag>
<diag><name>H66.92</name><desc>...left ear</desc></diag>
<diag><name>H66.93</name><desc>...bilateral</desc></diag>
</diag>

```

Output: A structured object:

```

code="H66.93",
description="Otitis media, unspecified, bilateral",
hierarchy_path="Diseases of the ear ? Diseases of middle ear ? Suppurative and unspecified otitis media ? ...
chapter="Diseases of the ear and mastoid process",
chapter_code="8",
includes=[], # Inherited from parent or empty
inclusion_terms=["Otitis media NOS", "Acute otitis media NOS"], # From parent H66.9
excludes1=[],
excludes2=[],
code_first=[],
use_additional_code=["code for any associated perforated tympanic membrane (H72.-)"],
parent_code="H66.9", # The parent in the hierarchy
children_codes=[], # H66.93 is a leaf ? no children
index_synonyms=[], # Filled in sub-step 1b
target_sex=[], # Filled in sub-step 1c
max_age_days=None,
)

```

Key decisions:

- ? Each becomes its own document ? no chunking needed
- ? The hierarchy path is accumulated during recursion (parent descriptions joined with " ? ")
- ? Parent-child relationships are explicitly stored for the Specificity Check later
- ? and are captured separately (both are synonyms but from different sources)

Sub-Step 1b: Parse the Index XML (Synonym Enrichment)

We parse the Index file to build a code ? synonyms mapping:

```

def parse_icd10_index(xml_path: str) -> dict[str, list[str]]:
    """Build: {"H66.93": ["Ear infection, middle, bilateral", ...]}"""

```

How it works:

For each , we build compound phrases by concatenating the path through nested elements:

```

mainTerm: "Diabetes, diabetic (mellitus) (sugar)"
??? term[1]: "with"
??? term[2]: "chronic kidney disease" ? code E11.22

```

Produces the synonym: "Diabetes, diabetic, with, chronic kidney disease" ? mapped to E11.22

After parsing the entire Index, we merge these synonyms into the Tabular objects:

```

def enrich_codes_with_synonyms(codes, code_synonyms, max_synonyms=10):
    """Merge index synonyms into each ICD10Code.index_synonyms field."""

```

Result for H66.93:

```

code="H66.93",
...
index_synonyms=["Otitis media, bilateral", "Ear infection, middle, bilateral"],
)

```

Why this matters: When a doctor says "ear infection in both ears," the embedding model needs to match this to H66.93. The Index provides exactly this mapping ? it's a free synonym dictionary built by medical coders who know what people actually search for.

Sub-Step 1c: Metadata Tagging

Apply demographic filters based on code prefixes:

```
SEX_RESTRICTIONS = {
    "O": "female",    # Pregnancy codes (000-09A)
    "N40": "male",    # Prostate codes
    "N41": "male",
    "N42": "male",
}

AGE_RESTRICTIONS = {
    "P": 28,          # Perinatal codes ? only for newborns (?28 days old)
}
```

Why: A 45-year-old male patient should NEVER see pregnancy codes (O-codes) in their suggestions. This filter is applied as a hard constraint during vector search, eliminating irrelevant results before scoring.

Sub-Step 1d: Export to JSONL

Each enriched ICD10Code object is serialized to one line of JSON:

```
{"code": "H66.93", "description": "Otitis media, unspecified, bilateral", "hierarchy_path": "Diseases of the e..."}
```

Output: data/icd10_processed/icd10_codes.jsonl ? approximately 72,000 lines.

Step 2: Embed & Index

Goal: Convert each code's text into a vector (dense) and a token frequency map (sparse), then store both in Qdrant for fast retrieval.

Files involved:

- ? src/mednote/rag/embeddings.py ? SapBERT embedding wrapper
- ? src/mednote/rag/indexer.py ? Qdrant collection setup + batch upsert
- ? scripts/build_index.py ? orchestration script

What Gets Embedded

Each code's to_embedding_text() produces:

```
H66.93: Otitis media, unspecified, bilateral
Hierarchy: Diseases of the ear ? Diseases of middle ear ? Suppurative and unspecified otitis media ? Otitis me...
Also known as: Otitis media NOS, Acute otitis media NOS, Ear infection, middle, bilateral
Excludes: ...
```

This combines three synonym sources for maximum retrieval signal:

- ? official included conditions from Tabular
- ? "also known as" names from Tabular
- index_synonyms ? natural-language terms from the Index file

Why SapBERT (Not a Generic Model)

Model	"heart attack" ? "acute myocardial infarction" ...
Generic (all-MiniLM-L6-v2)	~0.45 (low ? different words)
Generic (text-embedding-ada-002)	~0.55 (moderate)
SapBERT	~0.89 (high ? trained on UMLS medical synonym p...

SapBERT was specifically trained on the UMLS (Unified Medical Language System) to understand that medical synonyms should be close in embedding space. A generic model doesn't know that "MI" = "myocardial infarction" = "heart attack."

Dense + Sparse: Two Vectors Per Code

Each code gets stored with:

1. Dense vector (768 dimensions) ? SapBERT embedding of the full text
 - ? Good at: "heart attack" ? "acute myocardial infarction" (semantic similarity)
 - ? Bad at: "COPD" (an acronym, not naturally similar to anything)
2. Sparse vector (BM25 token frequencies) ? which words appear and how often
 - ? Good at: "COPD" matches exactly in codes that contain "COPD"
 - ? Bad at: "chest pain" ? "angina pectoris" (different words, same meaning)

What's Stored in Qdrant

Each of the ~72,000 codes becomes a point in Qdrant with:

```
Point {
  id: 12345,
  vector: [0.012, -0.034, ...],           // 768-dim SapBERT embedding
  sparse_vector: {bm25: {indices, values}}, // Token frequencies
  payload: {                             // Metadata (searchable + returnable)
    "code": "H66.93",
    "description": "Otitis media, unspecified, bilateral",
    "hierarchy_path": "Diseases of the ear ? ...",
    "includes": ["Otitis media NOS"],
    "inclusion_terms": ["Acute otitis media NOS"],
    "index_synonyms": ["Ear infection, middle, bilateral"],
    "target_sex": ["all"],                // For metadata filtering
    "max_age_days": null,
    "chapter_code": "8",
    "parent_code": "H66.9",
    "children_codes": [],
    "code_first": [],
    "use_additional_code": ["code for any associated perforated tympanic membrane (H72.-)"]
  }
}
```

Payload Indexes Created

We create indexes on filter fields for fast metadata filtering:

```
client.create_payload_index(field_name="target_sex", field_schema=KEYWORD)
client.create_payload_index(field_name="chapter_code", field_schema=KEYWORD)
```

This means the query WHERE target_sex IN ["all", "male"] executes at the index level, not by scanning all 72,000 points.

Step 3: Runtime Retrieval (Hybrid Search)

Goal: Given a clinical entity (e.g., "Acute bilateral otitis media"), find the 15 most relevant ICD-10 codes from the vector database.

File: src/mednote/rag/retriever.py

What Triggers This Step

The doctor types something ? Entity Extraction (Step 3 of the main pipeline) produces clinical terms:

```
Doctor input: "Kid has ear infection in both ears, mom's BP is high"
? (Entity Extraction via fast LLM)
Entities: ["Acute bilateral otitis media", "Essential hypertension"]
? (Each entity becomes a separate search query)
```

The Hybrid Search Formula

For each entity, we run:

Final Score = (Dense Score × 0.7) + (Sparse Score × 0.3)

Component	What it does	Weight	Why
Dense (SapBERT)	Finds semantically similar codes	0.7	Most medical queries are paraphrases
Sparse (BM25)	Finds exact keyword matches	0.3	Catches acronyms (COPD, STEMI, NOS)

Metadata Filter Applied

Before scoring, irrelevant codes are eliminated:

```
query_filter = Filter(must=[
    FieldCondition(key="target_sex", match=MatchAny(any=["all", patient_sex]))
])
```

For a 34-year-old female patient:

- ?? Codes with target_sex: ["all"] ? included
- ?? Codes with target_sex: ["female"] ? included (pregnancy codes are valid)
- ?? Codes with target_sex: ["male"] ? excluded (prostate codes impossible)

Output

Top 15 candidates per entity, sorted by hybrid score:

```
[
  {"code": "H66.93", "description": "Otitis media, unspecified, bilateral",
   "score": 0.91, "hierarchy": "...", "children_codes": [], ...},
  {"code": "H65.93", "description": "Nonsuppurative otitis media, bilateral",
   "score": 0.87, ...},
  {"code": "H66.9", "description": "Otitis media, unspecified",
   "score": 0.82, "children_codes": ["H66.90", "H66.91", "H66.92", "H66.93"], ...},
  ... (12 more)
]
```

Why 15 (Not 3 or 50)?

- ? 3 is too few ? the correct answer might be ranked #5 by the vector search alone
- ? 50 is too many ? the cross-encoder re-ranker (Step 4) is slow; scoring 50 items takes too long
- ? 15 is the sweet spot ? wide enough to catch the right answer, narrow enough to re-rank in <200ms

Step 4: Re-Ranking (Cross-Encoder Precision)

Goal: Take the 15 candidates and score each one directly against the original transcript, keeping only the top 3.

File: src/mednote/rag/reranker.py

Why Re-Ranking Is Needed

Vector search finds the right neighborhood but isn't great at picking the best result within it. Consider:

```
Transcript: "Patient has ear pain in both ears, fever, discharge"

Vector Search (Step 3) returns:
#1: H66.93 - Otitis media, unspecified, bilateral      ? relevant
#2: H60.93 - Otitis externa, bilateral                ? wrong (external ear, not middle)
#3: H65.93 - Nonsuppurative otitis media, bilateral   ? relevant
#4: H66.003 - Acute suppurative otitis media, bilateral ? MOST relevant (discharge = suppurative)
```

The vector embeddings don't know that "discharge" ? "suppurative." But the cross-encoder reads both texts together and understands this relationship.

How the Cross-Encoder Works

Unlike the bi-encoder (SapBERT) which embeds query and document separately, the cross-encoder reads both simultaneously:

```
Input pair: ("Patient has ear pain in both ears, fever, discharge",
            "H66.003: Acute suppurative otitis media without spontaneous rupture of ear drum, bilateral")
?
Cross-Encoder processes both texts together (attention between them)
?
Output: relevance score 0.94
```

It does this for all 15 candidates, then sorts by score and keeps the top 3.

The Speed Trade-off

Step	Documents scored	Time	Precision
Vector Search (Step 3)	72,000	~50ms	Moderate
Cross-Encoder (Step 4)	15	~200ms	High

If we ran the cross-encoder on all 72,000 codes, it would take ~16 minutes. By first narrowing to 15 with the fast vector search, then applying the precise cross-encoder, we get the best of both worlds.

Output

Top 3 codes with high-confidence relevance scores:

```
[
  {"code": "H66.003", "description": "Acute suppurative otitis media..., bilateral",
   "rerank_score": 0.94, "hierarchy": "...", ...},
  {"code": "H66.93", "description": "Otitis media, unspecified, bilateral",
   "rerank_score": 0.88, ...},
  {"code": "H65.93", "description": "Nonsuppurative otitis media, bilateral",
   "rerank_score": 0.81, ...},
]
```

Zero-Hit Fallback

If the highest re-rank score is below 0.7 (our confidence threshold), the system doesn't force a suggestion:

"Insufficient data to suggest an accurate ICD-10 code. Please manually assign in EHR."

This prevents the system from suggesting a wrong code just because it's the "least bad" match.

Step 5: Specificity Check

Goal: If any of the top 3 codes are "parent" codes with more specific children, surface those children so the doctor can select the most precise code.

File: src/mednote/rag/specificity.py

Why Specificity Matters

Insurance companies and auditors want the most specific code possible. Billing with a vague "unspecified" code when the information exists for a specific one causes:

Problem	Example	Consequence
Claim denial	Using H66.9 ("unspecified ear") when you know	Insurance refuses to pay
Lower reimbursement	Unspecified codes often pay less	Clinic loses revenue
Audit flag	Pattern of unspecified codes triggers audit	Time-consuming review process

How It Works

For each of the top 3 codes, check if children_codes is non-empty:

```
def check_and_expand(self, top_codes: list[dict]) -> list[dict]:
    for code_result in top_codes:
        children = code_result.get("children_codes", [])
        if children:
            # Fetch child details from Qdrant
            child_results = self._fetch_children(children)
            code_result["specificity_options"] = child_results
            code_result["needs_specificity"] = True
```

Example

```
Top code: H66.9 (Otitis media, unspecified)
? Has children! Expanding:

???? H66.90 - Otitis media, unspecified, unspecified ear
???? H66.91 - Otitis media, unspecified, right ear
???? H66.92 - Otitis media, unspecified, left ear
???? H66.93 - Otitis media, unspecified, bilateral ? transcript says "both ears"!
```

The system surfaces these options to the doctor:

Specificity suggestion: The transcript mentions bilateral involvement. Consider:

- H66.93 (bilateral) for greater specificity

- H66.91 (right ear) or H66.92 (left ear) if only one side is affected

When Specificity Check Does Nothing

If the top code is already a leaf (no children), it passes through unchanged:

```
Top code: H66.93 (Otitis media, unspecified, bilateral)
? children_codes: [] (empty ? already most specific)
? needs_specificity: False
? Pass through as-is
```

How It All Connects

Complete Flow for: "Kid has ear infection in both ears"

1. Doctor types: "Kid has ear infection in both ears"
2. Entity Extraction (fast LLM):
 - ? ["Acute bilateral otitis media"]
3. Patient Context (from EHR):
 - ? {age: 5, sex: "male"}
4. Hybrid Search in Qdrant:
 - Metadata filter: target_sex IN ["all", "male"]
 - Dense search (SapBERT): "Acute bilateral otitis media" ? H66.93 description
 - Sparse search (BM25): "bilateral" matches index_synonyms containing "bilateral"
 - Combined top 15 results
5. Cross-Encoder Re-Rank (against full transcript):
 - Scores all 15 against "Kid has ear infection in both ears"
 - Top 3: H66.93 (0.94), H65.93 (0.87), H66.9 (0.82)
6. Specificity Check:
 - H66.93: no children ? pass through
 - H65.93: no children ? pass through
 - H66.9: HAS children ? expand with laterality options
7. Final output to Note Generation:
 - Suggested codes:
 - H66.93 - Otitis media, unspecified, bilateral (Pending Physician Confirmation)
 - H65.93 - Nonsuppurative otitis media, bilateral (Pending Physician Confirmation)
 - H66.9 - Otitis media, unspecified (Consider: H66.93 for bilateral specificity)

Why Each Step Exists

Step	What would go wrong without it
Index synonym enrichment	"Ear infection" wouldn't match "Otitis media" a...
Metadata filtering	5-year-old boy might see pregnancy codes
Dense search	Acronyms like "COPD" would be missed
Sparse search	Paraphrases like "heart attack" ? MI would be m...
Cross-encoder re-rank	"Discharge" ? "suppurative" connection would be...
Specificity check	Doctor would get "unspecified" codes when speci...
Zero-hit fallback	System would force a wrong code rather than adm...

The Complete Data Journey

```
Official CMS XML (18.5 MB)
? ETL: parse + enrich + tag
Structured JSONL (72,000 documents)
```


? Embed: SapBERT + BM25 sparse
Qdrant Vector DB (72,000 points × 2 vectors each)
? Query: hybrid search + metadata filter
Top 15 candidates
? Re-rank: cross-encoder precision scoring
Top 3 codes (with confidence scores)
? Specificity: expand parent codes
Final suggestions (with specificity options)
? Inject into LLM prompt
SOAP note with cited, grounded ICD-10 suggestions

Every code the doctor sees can be traced backwards through this chain ? from the SOAP note, to the re-ranker score, to the vector search hit, to the specific JSONL entry, to the exact element in the official CMS XML. This is the audit trail required in healthcare.